Intune Compliance-Failure Diagnostics & Remediation — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Intune Compliance-Failure Diagnostics & Remediation (Government / GCC High) 5

The problem . . . 5

The solution . . . 5

What's different on GCC High (the parity gaps that matter) . . . 5

Business value . . . 5

Bottom line . . . 6

1. Solution Overview — Intune Compliance-Failure Diagnostics & Remediation (Government / GCC High) 6

What this solution does . . . 6

The deployment model (all solutions) . . . 6

Commercial vs GCC High — what's different . . . 7

What's inside . . . 8

Who this is for . . . 8

2. Requirements Guide — Compliance-Failure Diagnostics (Government / GCC High) 8

R1. A GCC High (or DoD) tenant on the US Government cloud . . . 8

R2. US Government licensing that includes Intune . . . 8

R3. Administrative roles (assigned in the Gov portals) . . . 9

R4. Microsoft Graph PowerShell configured for US Gov . . . 9

R5. Device network access to the US Gov service endpoints (no TLS break) . . . 9

R6. A supported, enrolled Windows baseline . . . 10

R7. A pilot/target security group (created in `portal.azure.us`) . . . 10

R8. Feature-availability check for the rules you're diagnosing . . . 10

Global Requirements — Intune Solutions (category-level) 10

G1. Microsoft Entra ID tenant . . . 11

G2. Microsoft Intune subscription . . . 11

G3. Administrative roles . . . 11

G4. Device enrollment in Intune . . . 11

G5. Pilot security group . . . 11

G6. Supported Windows baseline . . . 11

G7. Network access to Microsoft cloud endpoints . . . 11

3. Diagnostic Playbook — Compliance-Failure Diagnostics & Remediation (Government / GCC High) 12

Step 0 — Assess: capture the exact symptom (read-only) . . . 12

Branch A — "Not evaluated" or "No policy assigned" . . . 12

Branch B — "In grace period" . . . 13

Branch C — "Not compliant" on a specific setting (the common case) . . . 13
Branch D — "Compliant in Intune, but the user is still blocked" . . . 15
Branch E — Device missing, stale, or last check-in is old . . . 15
Step Final — Pilot → Validate → Expand every change . . . 16
5. Testing Guide — Compliance-Failure Diagnostics (Government / GCC High) 16
Test 1 — You are in the correct (sovereign) cloud . . . 16
Test 2 — Endpoint reachability and no TLS interception . . . 17
Test 3 — Discovery snapshot is accurate . . . 17
Test 4 — Report matches the portal population . . . 17
Test 5 — A single fix turns a pilot device Compliant . . . 17
Test 6 — Remediation honors `-WhatIf` by default . . . 18
Test 7 — Scope targeting preview is safe . . . 18
Test 8 — Conditional Access unblocks after Compliant . . . 18
Test 9 — Parity-gap rules are handled, not "fixed forever" . . . 18
6. Troubleshooting Guide — Compliance-Failure Diagnostics (Government / GCC High) 18
Scripts return nothing / "no devices found" . . . 19
`Connect-MgGraph` fails to authenticate on GCC High . . . 19
A portal URL won't load or bounces to a sign-in loop . . . 19
Device stuck at "Not evaluated" no matter what you assign . . . 19
BitLocker/Secure Boot/Code Integrity rule fails on a healthy device . . . 19
"Device threat level" rule never evaluates . . . 19
Device isn't compliant right after provisioning . . . 20
A device from the old tenant never appears in GCC High . . . 20
Compliant in Intune, still blocked from Microsoft 365 . . . 20
Remediation changed something you didn't expect . . . 20
Report counts don't match the portal . . . 20
Endpoints that worked last month now fail . . . 20
7. Best Practices — Compliance-Failure Diagnostics (Government / GCC High) 21
Sovereign-cloud hygiene . . . 21
Diagnose before you change . . . 21
Safety and change control . . . 21
Grace periods and Conditional Access . . . 22
Evidence and least privilege . . . 22
Documentation discipline . . . 22
8. Deliverables Summary — Compliance-Failure Diagnostics (Government / GCC High) 22
Documents . . . 22

PowerShell scripts (US Gov Graph) 23
How they work together 24
Success criteria 24
Appendix A — PowerShell Scripts 26

Executive Summary — Intune Compliance-Failure Diagnostics & Remediation (Government / GCC High)

The problem

On a **US sovereign-cloud (GCC High)** tenant, an Intune device that reports **Not compliant** — or **Not evaluated**, or **In grace period** — blocks Conditional Access and locks the user out of Microsoft 365. Diagnosing that failure on GCC High is **not** the Commercial process with a different sign-in: the admin portals, service endpoints, and Microsoft Graph all live on `*.us` domains, and several compliance-relevant capabilities behave differently from — or are absent versus — the Commercial cloud. Follow Commercial steps and the fixes fail **silently**: you sign into the wrong portal, allow the wrong endpoint through the proxy, or script against the Commercial Graph, and the device never turns compliant.

The solution

A **symptom-driven Diagnostic Playbook** plus five **US Gov Graph** PowerShell scripts that take a novice-to-intermediate admin from *the symptom the user reports* to *the exact GCC High blade and safe fix* — with every Commercial-vs-GCC High difference called out where it changes the diagnosis:

- **Diagnostic Playbook (symptom tree).** Branches for **Not evaluated**, **In grace period**, **Not

compliant **(per-setting: BitLocker/Secure Boot/Code Integrity, OS version, Defender AV, device threat level/MTD, password/encryption, custom rules),** Compliant-but-blocked **(Conditional Access), and** stale/missing** devices — each ending in a GCC High blade and a validation step.

- **Five scripts, all on US Gov Graph** (`Connect-MgGraph -Environment USGov`): Discovery, Validation,

Report (read-only), and Remediation + Set-ScopeTargeting (**default** `-WhatIf`).

- **Cited GCC High parity reference** documenting every difference from Microsoft's own US Government

documentation, with anything unconfirmed flagged **"verify for GCC High"** — never invented.

What's different on GCC High (the parity gaps that matter)

- **All portals/endpoints are** `*.us` — `intune.microsoft.us`, `portal.azure.us`,

`login.microsoftonline.us`, `graph.microsoft.us`; devices check in via `*.manage.microsoft.us` / `*.dm.microsoft.us`, which **must not be SSL-inspected**.

- **Feature gaps that change a diagnosis:** classic **Windows Autopilot** unavailable; **Cloud PKI**

and **Remote Help** planned-not-available; **MTD** works only for vendors present in Gov; **some newer DHA capabilities** planned (core BitLocker/Secure Boot/Code Integrity rules do work); most **Windows update workflows** unavailable; **no in-place migration** from Commercial.

- **Scripting must use** `-Environment USGov` or it silently queries the Commercial cloud.

Business value

• **Faster time-to-compliant.** A repeatable symptom→fix path replaces trial-and-error on an unfamiliar sovereign cloud, restoring Microsoft 365 access blocked by Conditional Access.

• **No silent failures.** The right portals, endpoints, and Graph environment are baked in, so fixes actually land instead of appearing to.

• **Safe by design.** Read-only discovery, preview-first remediation (`-WhatIf`), and a mandatory pilot prevent a compliance change from blocking an entire fleet at once.

• **Audit-ready.** Cited parity notes and before/after script evidence support a FedRAMP High-accredited (GCC High/DoD) operating posture.

Bottom line

Intune compliance failures diagnosed and remediated correctly on GCC High — right `*.us` portals, right endpoints, parity gaps handled — proven on a pilot before you scale.

1. Solution Overview — Intune Compliance-Failure Diagnostics & Remediation (Government / GCC High)

This is the **Government (GCC High) edition**. It diagnoses and remediates **Intune device compliance failures** on a **US sovereign-cloud** tenant — where the admin portals, service endpoints, and Microsoft Graph all live on `*.us` domains, and where a handful of compliance-relevant features behave differently from, or are absent versus, the Commercial cloud. Following Commercial steps on GCC High produces *silent* failures: you sign into the wrong portal, allow the wrong endpoint through the proxy, or script against the Commercial Graph — and devices never turn compliant.

What this solution does

A device that reports **Not compliant** (or **Not evaluated**, or **In grace period**) blocks Conditional Access and locks the user out of Microsoft 365. This solution gives a novice admin a **symptom-driven Diagnostic Playbook** (section 3) that walks from the symptom the user reports to the exact GCC High portal blade, the exact setting, and a safe fix — plus five PowerShell scripts that connect to **US Gov Graph** (`Connect-MgGraph -Environment USGov`) to discover, validate, report, and (with `-WhatIf` by default) remediate.

The scope is **diagnostics and remediation of existing compliance policies**, not authoring a compliance baseline from scratch. You already have compliance policies; this fixes why devices fail them on GCC High.

The deployment model (all solutions)

Assess → **Configure** → **Pilot** → **Validate** → **Expand.** Here that maps to: **Assess** the reported symptom and the tenant's compliance surface (Discovery/Report scripts) → **Configure** the fix in the correct GCC High blade → **Pilot** on one affected device or a small group → **Validate** the device turns compliant (Validation script + portal) → **Expand** the fix to the affected population.

Commercial vs GCC High — what's different

GCC High is **not** the Commercial portal with a different sign-in. The differences below are the ones that change how you *diagnose* compliance. Every one is drawn from Microsoft's own US Government documentation (cited in `gcc-high-parity-notes.md`); anything Microsoft does not explicitly confirm for GCC High is flagged **"verify for GCC High"** rather than assumed.

1. Different admin portals and service URLs — all `*.us`. Using a `.com` portal or endpoint on a GCC High tenant causes authentication failures, so a device or admin "silently" never reaches the service.

Purpose	Commercial	**GCC High**
Intune admin center	`intune.microsoft.com`	`intune.microsoft.us`
Entra admin center	`entra.microsoft.com`	`portal.azure.us` (Microsoft Entra ID blade)
Microsoft 365 admin	`admin.microsoft.com`	`portal.office365.us`
Defender portal	`security.microsoft.com`	`security.microsoft.us`
Purview / Compliance	`compliance.microsoft.com`	`compliance.microsoft.us`
Company Portal (web)	`portal.manage.microsoft.com`	`portal.manage.microsoft.us`
Sign-in (Evo STS)	`login.microsoftonline.com`	`login.microsoftonline.us`
Microsoft Graph	`graph.microsoft.com`	`graph.microsoft.us`

2. Devices reach a different set of service endpoints. Compliance check-in, enrollment, and reporting flow through `*.manage.microsoft.us` and `*.dm.microsoft.us` — and SSL/TLS inspection is **not supported** on `*.manage.microsoft.us` (or on the Defender/EPM `*.dm.microsoft.us` endpoints). A proxy that breaks TLS on those is one of the most common causes of a device stuck at **Not evaluated** in GCC High. Section 2 lists the endpoints; section 3 turns them into a symptom.

3. Some compliance-relevant features are absent or lag. The features that most often change a GCC High compliance diagnosis:

- **Classic Windows Autopilot is not available** in GCC High (only Autopilot **device preparation**, with some modes still planned) — so "device isn't compliant right after provisioning" has a different root cause than in Commercial.
- **Cloud PKI** (GCC High) and **Remote Help** are **planned, not yet available** — certificate-based compliance and assisted remediation workflows differ.
- **Mobile Threat Defense (MTD)** connectors work **only for vendors that also support GCC High** — a "device risk level" compliance rule can be un-evaluatable if your MTD vendor isn't in GCC High.
- **Some newer Windows Device Health Attestation (DHA) capabilities** are listed as planned for GCC High; the core BitLocker / Secure Boot / Code Integrity compliance settings **do** use the DHA service across Commercial, GCC High, and DoD. This nuance is called out precisely in section 3 and `gcc-high-parity-notes.md` — **verify for GCC High** before promising a specific DHA-backed rule.

4. Scripting must target US Gov Graph. Every script in this pack connects with `Connect-MgGraph -Environment USGov` (use `-Environment USGovDoD` for DoD). Omit `-Environment` and the module authenticates against the Commercial endpoint and returns nothing — the single most common scripting failure on GCC High.

What's inside

- **This overview**, an **executive summary**, a **requirements guide** (section 2), the

Diagnostic Playbook (section 3), a **testing guide** (section 5), a **troubleshooting guide** (section 6), **best practices** (section 7), a **deliverables summary** (section 8), and a cited **GCC High parity reference** (`gcc-high-parity-notes.md`).

- **Five PowerShell scripts** — Discovery, Validation, Report (all read-only), Remediation and

Set-ScopeTargeting (both default to `-WhatIf`), all on US Gov Graph.

Who this is for

A novice-to-intermediate admin operating a GCC High tenant who needs to turn **Not compliant** into **Compliant** correctly — using the right sovereign portals and endpoints, with the Commercial-vs-GCC High gaps called out so nothing fails silently.

2. Requirements Guide — Compliance-Failure Diagnostics (Government / GCC High)

Confirm every requirement below **before** diagnosing. The category-level prerequisites (`global-requirements-intune-solutions.md`) still apply — but on GCC High you must confirm them using the `*.us` **sovereign portals**, not the Commercial `.com` ones. Each entry: **Description · Why · Verify · Implement-if-missing.** Anything Microsoft does not explicitly confirm for GCC High is marked **"verify for GCC High."**

> ***Golden rule:*** *if any URL you open, allow, or script against ends in `.com`, stop — on GCC High it should almost always end in `.us`. See the portal map in section 1.*

R1. A GCC High (or DoD) tenant on the US Government cloud

Description: The tenant is provisioned in the Intune **government cloud** (GCC High / DoD, IL4/IL5), not Commercial or GCC (which shares the Commercial Intune instance). **Why:** Every portal, endpoint, and Graph call in this pack targets `*.us`; on a Commercial tenant they won't resolve to your data. **Verify:** You sign in at `https://intune.microsoft.us` and the admin center loads with your tenant. **Tenant administration** → **Tenant status** shows an active Intune subscription. **Implement:** GCC High is purchased through an authorized government/agency licensing path; it cannot be converted from a Commercial tenant in place (devices must unenroll and re-enroll). Engage your licensing provider.

R2. US Government licensing that includes Intune

Description: A Gov plan that bears Intune — e.g. **Microsoft 365 G3 / G5 (GCC High)**, **Enterprise Mobility + Security G3/G5**, or standalone **Intune Plan 1 for Government**. **Why:** Compliance policy

evaluation and Conditional Access require Intune (and Entra ID P1/P2 for CA). **Verify:** `portal.office365.us` → **Billing** → **Licenses**; confirm a Gov SKU that includes Intune and Entra ID P1 or higher. **Implement:** Assign the Gov SKU to affected users.

R3. Administrative roles (assigned in the Gov portals)

Description: The operator holds **Intune Administrator** (and read access to Entra sign-in / CA). **Why:** Editing compliance policies, assignments, and Conditional Access requires elevated rights. **Verify:** `portal.azure.us` → **Microsoft Entra ID** → **Roles and administrators** → confirm **Intune Administrator** (and, for CA changes, **Conditional Access Administrator** or **Security Administrator**). **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes. For the scripts, the same account signs in via `Connect-MgGraph -Environment USGov`.

R4. Microsoft Graph PowerShell configured for US Gov

Description: The **Microsoft.Graph** PowerShell modules, used with `-Environment USGov` (or `USGovDoD` for DoD). **Why:** Without `-Environment`, the module authenticates against **Commercial** Graph and returns nothing for a GCC High tenant — the No. 1 scripting failure on GCC High. **Verify:** In PowerShell: `Get-MgEnvironment` lists **USGov** (graph = `https://graph.microsoft.us`, login = `https://login.microsoftonline.us`) and **USGovDoD** (graph = `https://dod-graph.microsoft.us`). Then `Connect-MgGraph -Environment USGov -Scopes "DeviceManagementManagedDevices.Read.All"` succeeds. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. The built-in Microsoft Graph PowerShell app is supported for USGov/USGovDoD (no custom app registration required); other sovereign clouds do require a custom app — **verify for GCC High** if your tenant is in a different sovereign cloud.

R5. Device network access to the US Gov service endpoints (no TLS break)

Description: Managed devices reach the `*.us` Intune service endpoints for enrollment, compliance check-in, and reporting — **without SSL/TLS inspection** on the endpoints where Microsoft prohibits it. **Why:** A device that cannot check in (or whose TLS is broken by a proxy) sits at **Not evaluated** or goes **stale**, which reads as non-compliant to Conditional Access. **Verify:** From an affected device, confirm reachability of the endpoints below; confirm your proxy is **not** doing TLS interception on `*.manage.microsoft.us` or `*.dm.microsoft.us`. **Implement:** Allow these through the proxy/firewall (HTTP **80** and HTTPS **443**), unauthenticated where noted, and **exclude them from SSL inspection**:

Endpoint (GCC High / US Gov)	Used for
`*.manage.microsoft.us`	Intune enrollment, compliance check-in, reporting. **No SSL inspection.** Needs *unauthenticated* proxy access for some tasks (e.g. software-update downloads).
`*.dm.microsoft.us`	Defender for Endpoint security-settings-management and Endpoint Privilege Management check-in/reporting. **No SSL inspection.**
`login.microsoftonline.us`	Sign-in / token issuance (Evo STS).
`enterpriseregistration.microsoftonline.us`	Entra device registration (ADRS).

Endpoint (GCC High / US Gov)	Used for
`graph.microsoft.us` (DoD: `dod-graph.microsoft.us`)	Microsoft Graph — used by these scripts and by the portal.
`portal.manage.microsoft.us`	Company Portal (web) for end users.
`has.spserv.microsoft.com`	Health/attestation service dependency. **No SSL inspection.** (This one is a `.com` host by Microsoft's design — verify against the current Gov endpoints list.)
`*.events.data.microsoft.com`	Client-health diagnostic data.

> *Endpoints and IPs change as the service scales; always re-confirm against Microsoft's live **"Network endpoints for US government deployments — Microsoft Intune"** list (cited in `gcc-high-parity-notes.md`) — the wildcards exist precisely because the underlying hosts rotate.*

R6. A supported, enrolled Windows baseline

Description: Windows 10 21H2+ or Windows 11, current updates, **enrolled** in the GCC High tenant. **Why:** Compliance only evaluates enrolled, checked-in devices on a supported OS. **Verify:** On the device, `winver`; in `intune.microsoft.us` → **Devices** → **All devices**, the device appears with a recent **Last check-in**. **Implement:** Enroll/re-enroll the device in the GCC High tenant (there is **no** in-place migration from Commercial — unenroll then re-enroll). Patch to a supported build.

R7. A pilot/target security group (created in `portal.azure.us`)

Description: An Entra **security group** of the affected (or pilot) users/devices. **Why:** Fixes are piloted and assigned by group, never tenant-wide first. **Verify:** `portal.azure.us` → **Microsoft Entra ID** → **Groups**; the group exists. **Implement: Groups** → **New group** → **Security**; add the affected members. `Set-ScopeTargeting.ps1` can preview and apply a policy's group assignment (default `-WhatIf`).

R8. Feature-availability check for the rules you're diagnosing

Description: Confirm the specific compliance rule you're troubleshooting is a **supported** feature in GCC High before assuming a misconfiguration. **Why:** A rule that depends on a feature **absent** in GCC High (e.g. an MTD vendor not present in Gov, or a planned-but-unavailable capability) can never evaluate — that's a parity gap, not a bug. **Verify:** Cross-check the rule against `gcc-high-parity-notes.md` and Microsoft's **Intune Government service** description. **Implement:** If the feature is absent/planned in GCC High, remove or replace the rule (section 3 covers the substitutions); do not spend hours "fixing" a rule the sovereign cloud cannot evaluate.

Deployment model: Assess → **Configure** → **Pilot** → **Validate** → **Expand.** Do not skip Pilot — a compliance change that flips a whole fleet to **Not compliant** blocks every user at once through Conditional Access.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Diagnostic Playbook — Compliance-Failure Diagnostics & Remediation (Government / GCC High)

This is a **symptom-driven** playbook. Start from what the user or the portal reports, follow the branch, apply the fix in the **correct GCC High blade**, then validate. Every portal path uses the `*.us` sovereign URLs — if you ever land on a `.com` portal, you are in the wrong cloud and the device you're looking at isn't there.

> ***Work in*** `https://intune.microsoft.us` *(Intune admin center) and* `https://portal.azure.us` *(Entra ID). Scripts connect with* `Connect-MgGraph -Environment USGov` *(DoD:* `-Environment USGovDoD`*).*
>
> ***GCC High note (read first):*** *before deep-diving a rule, confirm the rule's feature is actually* ***supported*** *in GCC High (see the ■ markers below and* `gcc-high-parity-notes.md`*). A rule that depends on an absent/planned Gov feature can never evaluate — that's a parity gap, not a misconfiguration.*

Step 0 — Assess: capture the exact symptom (read-only)

1. In `intune.microsoft.us` → **Devices** → **All devices**, open the affected device → **Device compliance. Note the** overall state **and the** per-policy / per-setting** state.

1. Run `INTCOMPDXG-Discovery.ps1` (US Gov Graph) to snapshot the device: enrollment state, last check-in time, which compliance policies apply, and each failing setting.

1. Run `INTCOMPDXG-Report.ps1` to see the failure across the affected group (is this one device or a pattern?).

Match the **overall state** to a branch:

Symptom (state)	Go to
Not evaluated / **No compliance policies assigned**	Branch A
In grace period	Branch B
Not compliant (a specific setting fails)	Branch C
Compliant in Intune, but user still blocked	Branch D
Device **missing / stale / last check-in old**	Branch E

Branch A — "Not evaluated" or "No policy assigned"

The device isn't being measured at all. In GCC High this is very often an **endpoint / TLS** problem, not a policy problem.

Decision tree

1. **Is a compliance policy actually assigned to this device's group?**

• `intune.microsoft.us` → **Devices** → **Compliance** → **Policies** → open the policy → **Assignments**.

• If no policy targets the device's group → assign one (or use `INTCOMPDXG-Set-ScopeTargeting.ps1`,

default `-WhatIf`, to preview then apply the group assignment). Done.

1. **Is the "mark devices with no compliance policy" tenant default set to *Not compliant*?**

• **Devices** → **Compliance** → **Compliance policy settings** → **Mark devices with no compliance policy

assigned as → **for CA to work this is usually** Not compliant**. Note the** compliance status validity period** here too (Branch B).

1. **Can the device reach the GCC High service endpoints? ■ GCC High-specific**

• On the device confirm reachability of `*.manage.microsoft.us` and `login.microsoftonline.us`.

• **Confirm the proxy is NOT doing SSL/TLS inspection** on `*.manage.microsoft.us` (and

`*.dm.microsoft.us` for Defender/EPM). TLS interception there breaks check-in silently — classic GCC High cause of **Not evaluated**. Fix the proxy exclusion (section 2, R5), then force a sync.

1. **Force a check-in and re-evaluate:** device **Settings → Accounts → Access work or school → Info

→ Sync**, or** Sync** from the device page in `intune.microsoft.us`. Re-run Discovery.

Fix → Validate: after assignment/endpoint fix, run `INTCOMPDXG-Validation.ps1` and confirm the device moves to **Compliant** (or to a specific failing setting → Branch C).

Branch B — "In grace period"

The device failed a rule but is inside the configured grace window; it will flip to **Not compliant** when the window expires.

1. Open the device → **Device compliance** → identify **which policy** granted grace and **which

setting** is failing.

1. Treat the failing setting as **Branch C** (fix the underlying setting now, before the window

closes).

1. Review the grace period itself: policy → **Actions for noncompliance** → the **"Mark device

noncompliant"** schedule (in days). A grace period that's too long hides real failures; too short blocks users mid-remediation. Adjust deliberately, then Pilot.

Branch C — "Not compliant" on a specific setting (the common case)

Read the **failing setting name** from the device's **Device compliance** detail, then jump to the matching sub-branch. Each shows: what it checks, the GCC High nuance, where to fix it, and how to validate.

C1 — BitLocker / Secure Boot / Code Integrity (Device Health)

- **Checks:** the Windows compliance rules **Require BitLocker**, **Require Secure Boot**, **Require

code integrity **— all evaluated via the** Device Health Attestation (DHA)** service.

- ■ **GCC High nuance:** the **core** BitLocker / Secure Boot / Code Integrity rules **do** use the

DHA service across Commercial, **GCC High**, and DoD. However, Microsoft lists **some newer Windows DHA capabilities as *planned* (not yet available)** in GCC High — so a rule that depends on an expanded DHA capability may not evaluate. **Verify for GCC High** which specific DHA-backed attribute your rule uses before promising it works (see `gcc-high-parity-notes.md`).

- **Diagnose on the device:** run `INTCOMPDXG-Discovery.ps1` — it reports TPM readiness, Secure

Boot state, and BitLocker OS-drive protection. Common root causes: device hasn't **rebooted** since encryption; firmware/TPM out of date; the device can't reach the attestation dependency (`has.spserv.microsoft.com`, **no SSL inspection**).

- **Fix:** encrypt the OS drive (or fix the DHA/attestation reachability), **reboot**, force a sync.
- **Validate:** `INTCOMPDXG-Validation.ps1` → the setting reports PASS and the device turns Compliant.

C2 — Minimum OS version / OS build

- **Checks:** the device's OS build meets the policy's **Minimum OS version**.
- **Fix:** patch the device. Note ■ that several **Windows update features** (feature/quality/expedite/

driver updates, Autopatch) are **not available in GCC High**, so you cannot lean on those Intune update workflows to remediate — use your available patching mechanism, then re-sync.

- **Validate:** `winver` on the device; re-run Validation.

C3 — Antivirus / antispyware / Microsoft Defender real-time protection

- **Checks:** Defender is on, signatures current, real-time protection enabled.
- ■ **GCC High nuance: Defender for Endpoint security-settings-management is supported** in GCC

High and DoD (you can manage Defender on Defender-onboarded devices via Intune). Confirm the device reaches `*.dm.microsoft.us` (**no SSL inspection**) for Defender check-in.

- **Fix:** enable/repair Defender; confirm signature update path; force sync.
- **Validate:** Discovery reports Defender state; device turns Compliant.

C4 — Device threat level (Mobile Threat Defense)

- **Checks:** the device's **risk level** from an MTD connector is at/below the allowed threshold.
- ■ **GCC High nuance (critical):** **MTD connectors work only for vendors that ALSO support GCC

High. **If your MTD vendor isn't present in Gov, this rule** cannot be evaluated **and the device may be stuck. In** `intune.microsoft.us` → Tenant administration → Connectors and tokens** you only see the

connectors available in Gov.

• **Fix:** if no GCC High MTD connector exists for your vendor, **remove or replace** the "device threat level" rule (parity gap, not a bug). If a supported connector exists, confirm it's connected and the device is enrolled with the MTD app.

• **Validate:** re-run Report; the rule now evaluates (or is removed).

C5 — Password / passcode / encryption (mobile) and jailbreak/root

• **Checks:** password complexity, encryption, jailbreak/root status per platform.

• ■ **GCC High nuance:** standard MDM compliance settings are supported; platform coverage matches Commercial (Windows, iOS/iPadOS, macOS, Android, AOSP, Linux GA). AOSP has some device restrictions — **verify for GCC High** if you rely on a specific AOSP capability.

• **Fix:** correct the setting on-device or via a configuration profile; force sync.

• **Validate:** Validation reports PASS.

C6 — Custom compliance (script-based) rules

• **Checks:** a custom compliance script + JSON evaluated on the device.

• **Diagnose:** confirm the detection script deploys and returns the expected JSON; a script that can't run (execution policy, missing dependency) reads as non-compliant.

• **Fix:** correct/redeploy the script; force sync. Re-run Discovery.

Branch D — "Compliant in Intune, but the user is still blocked"

The device is Compliant in `intune.microsoft.us`, yet the user is denied Microsoft 365. This is almost always **Conditional Access**, not compliance.

1. `portal.azure.us` → **Microsoft Entra ID** → **Security** → **Conditional Access** → **Sign-in logs** (or **Monitoring** → **Sign-in logs**) → open the blocked sign-in → **Conditional Access** tab → find the **policy that blocked** and the **grant control** ("Require device to be marked as compliant").

1. **Compliance-state propagation lag:** a device that *just* turned compliant may not have propagated to Entra yet. Have the user sign out/in; re-check after propagation.

1. **Platform mismatch:** the CA policy may target a platform the device reports differently. Confirm the CA policy's **Conditions** → **Device platforms** matches.

1. **Fix in the correct portal:** compliance rules in `intune.microsoft.us`; the **grant control** in `portal.azure.us` CA. Don't edit a compliance policy to fix a CA problem.

Branch E — Device missing, stale, or last check-in is old

Compliance is evaluated at check-in; a device that stopped checking in goes stale and reads non-compliant.

1. **Confirm enrollment:** `intune.microsoft.us` → **Devices** → **All devices**. If absent, the device isn't enrolled in **this** tenant. ■ Remember there is **no in-place migration** from Commercial to GCC High — a device enrolled in a Commercial tenant will never appear here; it must **unenroll then re-enroll** in the Gov tenant.

1. **Endpoint reachability (GCC High):** re-check `*.manage.microsoft.us` / `*.dm.microsoft.us` reachability and the **no-SSL-inspection** rule (section 2, R5). A device silently losing check-in is usually a proxy/TLS regression.

1. **Provisioning context: ■ classic Windows Autopilot is not available in GCC High** — if a device "wasn't compliant right out of provisioning," the Commercial Autopilot compliance timing guidance doesn't apply; use **Autopilot device preparation** (some modes still planned) or standard enrollment.

1. **Force sync / re-enroll** as needed; re-run Discovery and Report.

Step Final — Pilot → Validate → Expand every change

1. **Pilot:** apply the fix to **one affected device or a small group** first (`INTCOMPDXG-Set-ScopeTargeting.ps1`, default `-WhatIf`, to preview the assignment).

1. **Validate:** `INTCOMPDXG-Validation.ps1` on a pilot device + confirm **Compliant** in `intune.microsoft.us`; `INTCOMPDXG-Report.ps1` for the group.

1. **Remediate at scale (carefully):** `INTCOMPDXG-Remediation.ps1` runs `-WhatIf` **by default** — review the planned actions, then re-run with `-Execute` (or `-WhatIf:$false`) only after the pilot passes.

1. **Expand:** widen the assignment in waves; re-validate each wave. A compliance change that flips a fleet to **Not compliant** blocks every user at once through Conditional Access — never skip Pilot.

Order of operations

```
Capture symptom (Discovery/Report) → branch A–E → fix in the correct *.us blade → Pilot → Validate (Validation + portal) → Remediate (-WhatIf first) → Expand in waves
```

5. Testing Guide — Compliance-Failure Diagnostics (Government / GCC High)

Every test uses the `*.us` portals and US Gov Graph. A test only **passes** when the evidence is visible in `intune.microsoft.us` (and, for Conditional Access, `portal.azure.us`) — not just in a script's output. Run these on a **pilot device** before expanding any fix.

> *Connect scripts with* `Connect-MgGraph -Environment USGov` *(DoD:* `-Environment USGovDoD`*).*

Test 1 — You are in the correct (sovereign) cloud

- **Do:** open `https://intune.microsoft.us` and confirm your tenant loads. In PowerShell run `Connect-MgGraph -Environment USGov -Scopes "DeviceManagementManagedDevices.Read.All"` and then `(Get-MgContext).Environment`.
- **Pass:** portal is `intune.microsoft.us` (not `.com`) **and** `(Get-MgContext).Environment` returns **USGov** (or **USGovDoD**).
- **Fail:** any `.com` portal, or `Get-MgContext` shows `Global` → you're querying the Commercial cloud; reconnect with `-Environment USGov`.

Test 2 — Endpoint reachability and no TLS interception

- **Do:** from a pilot device, confirm reachability of `*.manage.microsoft.us`, `login.microsoftonline.us`, and (if using Defender/EPM) `*.dm.microsoft.us`. Confirm the proxy is **not** SSL-inspecting `*.manage.microsoft.us` / `*.dm.microsoft.us`.
- **Pass:** endpoints reachable on 80/443 and the device's **Last check-in** in `intune.microsoft.us` is recent (minutes, not days).
- **Fail:** endpoints blocked or TLS intercepted → device stalls at **Not evaluated** (fix per section 2, R5).

Test 3 — Discovery snapshot is accurate

- **Do:** run `INTCOMPDXG-Discovery.ps1` against a known-failing device.
- **Pass:** the script reports the same failing policy/setting that `intune.microsoft.us` → device → **Device compliance** shows.
- **Fail:** empty output usually means Commercial-cloud connection (Test 1) or no Graph scope consented.

Test 4 — Report matches the portal population

- **Do:** run `INTCOMPDXG-Report.ps1` for the affected group.
- **Pass:** the compliant/non-compliant counts match **Devices** → **Compliance** → **Overview** (and per-policy monitoring) in `intune.microsoft.us`.
- **Fail:** counts differ → check scope filters, the group targeted, and the environment.

Test 5 — A single fix turns a pilot device Compliant

- **Do:** apply one branch-C fix to one device; force **Sync**; wait for re-evaluation.
- **Pass:** `INTCOMPDXG-Validation.ps1` reports PASS for that setting **and** the device shows **Compliant** in `intune.microsoft.us`.

• **Fail:** still failing → confirm the device rebooted (for DHA-backed rules), checked in, and that the rule's feature is actually **supported in GCC High** (section 3 ■ markers).

Test 6 — Remediation honors `-WhatIf` by default

• **Do:** run `INTCOMPDXG-Remediation.ps1` with no execute flag.

• **Pass:** it **only previews** planned actions and changes nothing; a real change happens **only** when you re-run with `-Execute` (or `-WhatIf:$false`).

• **Fail:** if it changes state without an explicit execute flag, stop and review — that violates the safety contract.

Test 7 — Scope targeting preview is safe

• **Do:** run `INTCOMPDXG-Set-ScopeTargeting.ps1` with no execute flag.

• **Pass:** it prints the **planned** assignment change (add/remove group on the policy) and makes no change until re-run with `-Execute`.

• **Fail:** any assignment change without the execute flag → stop.

Test 8 — Conditional Access unblocks after Compliant

• **Do:** on a device that is now **Compliant**, have the user sign out/in to a CA-gated app.

• **Pass:** access is granted; `portal.azure.us` → **Sign-in logs** shows the CA policy **satisfied**.

• **Fail:** still blocked → Branch D (propagation lag, platform mismatch, or a different CA policy).

Test 9 — Parity-gap rules are handled, not "fixed forever"

• **Do:** for any rule tied to an absent/planned GCC High feature (e.g. MTD vendor not in Gov), confirm it is **removed or replaced**, not left permanently failing.

• **Pass:** no device is stuck non-compliant on a rule the sovereign cloud cannot evaluate.

• **Fail:** a rule references a Gov-unavailable feature → replace it (section 3 / parity notes).

Exit criteria: Tests 1–9 pass on the pilot; Discovery/Report match the portal; every scripted change previewed before execution; no device stuck on a GCC High parity gap. Only then expand the fix in waves.

6. Troubleshooting Guide — Compliance-Failure Diagnostics (Government / GCC High)

Failures specific to running compliance diagnostics on a US sovereign-cloud tenant. The **Diagnostic Playbook** (section 3) handles *device* compliance symptoms; this section handles the failures of the **tooling and environment** you use to diagnose them. All portals are `*.us`.

Scripts return nothing / "no devices found"

- **Cause:** connected to the **Commercial** cloud. `Connect-MgGraph` without `-Environment` defaults to Global.
- **Fix:** `Connect-MgGraph -Environment USGov` (DoD: `USGovDoD`). Confirm with `(Get-MgContext).Environment` = **USGov**. This is the single most common GCC High scripting failure.

`Connect-MgGraph` fails to authenticate on GCC High

- **Cause:** wrong environment value, unconsented scopes, or (for non-USGov/USGovDoD sovereign clouds) the built-in app isn't usable.
- **Fix:** verify `Get-MgEnvironment` lists **USGov** (`graph.microsoft.us`, `login.microsoftonline.us`) and **USGovDoD** (`dod-graph.microsoft.us`). For USGov/USGovDoD the built-in Microsoft Graph PowerShell app is supported; **other** sovereign clouds require a **custom app registration** created at `portal.azure.us` — **verify for GCC High** which cloud your tenant is in.

A portal URL won't load or bounces to a sign-in loop

- **Cause:** a `.com` portal URL on a GCC High tenant, or a stale Commercial bookmark.
- **Fix:** use the `*.us` equivalent (section 1 map). Clear cached Commercial creds; sign in at `login.microsoftonline.us`.

Device stuck at "Not evaluated" no matter what you assign

- **Cause (GCC High-specific):** the device can't reach `*.manage.microsoft.us`, **or** a proxy is **SSL-inspecting** it (unsupported).
- **Fix:** allow `*.manage.microsoft.us` (unauthenticated where required) and **exclude it from TLS inspection**; **same for** `*.dm.microsoft.us` **(Defender/EPM). Force** Sync**, re-run Discovery.

BitLocker/Secure Boot/Code Integrity rule fails on a healthy device

- **Cause:** device hasn't **rebooted** since encryption; firmware/TPM outdated; attestation dependency (`has.spserv.microsoft.com`) blocked or TLS-inspected; **or** the rule uses a newer DHA capability that is **planned, not yet available** in GCC High.
- **Fix:** reboot; update firmware; unblock/exclude the attestation endpoint from inspection. If the rule depends on an unavailable DHA capability, **verify for GCC High** and substitute (parity notes).

"Device threat level" rule never evaluates

• **Cause (GCC High-specific):** the **MTD vendor isn't present in GCC High** — MTD connectors work only for vendors that also support Gov.

• **Fix:** in `intune.microsoft.us` → **Tenant administration** → **Connectors and tokens**, confirm a Gov connector exists. If not, **remove or replace** the threat-level rule; don't try to "fix" it.

Device isn't compliant right after provisioning

• **Cause (GCC High-specific):** you expected **classic Windows Autopilot** timing — **classic Autopilot is not available in GCC High **(only Autopilot** device preparation**, with some modes planned).

• **Fix:** use Autopilot device preparation or standard enrollment; don't apply Commercial Autopilot compliance-grace guidance.

A device from the old tenant never appears in GCC High

• **Cause:** there is **no in-place migration** from Commercial to the government cloud.

• **Fix:** the device must **unenroll** from the Commercial tenant and **re-enroll** in the GCC High tenant. Until then it will never appear or evaluate here.

Compliant in Intune, still blocked from Microsoft 365

• **Cause:** Conditional Access, not compliance (propagation lag, platform mismatch, or a different CA policy).

• **Fix:** Branch D — `portal.azure.us` → **Sign-in logs** → the blocking CA policy + grant control.

Remediation changed something you didn't expect

• **Cause:** you ran it with an execute flag before piloting.

• **Fix:** Remediation and Set-ScopeTargeting default to `-WhatIf`. Always preview first; execute (`-Execute` / `-WhatIf:$false`) only after a pilot passes. Roll back via the policy's assignment/ setting history.

Report counts don't match the portal

• **Cause:** wrong environment, wrong group/scope filter, or comparing different time windows.

• **Fix:** confirm **USGov** environment, the exact group targeted, and compare against **Devices → Compliance → Overview** in `intune.microsoft.us`.

Endpoints that worked last month now fail

• **Cause:** the `*.us` wildcard hosts / IP ranges rotate as the service scales (and Intune adds Azure

Front Door ranges).

- **Fix:** re-pull Microsoft's live **US Government endpoints** list; allow the current ranges; never

pin stale IPs. Keep the wildcard allow-rules (`*.manage.microsoft.us`, `*.dm.microsoft.us`).

Still stuck? Re-run Step 0 of the Diagnostic Playbook (section 3), confirm you're in USGov + `*.us`, and confirm the failing rule's feature is actually **supported in GCC High** (`gcc-high-parity-notes.md`) before escalating — a parity gap is not a bug.

7. Best Practices — Compliance-Failure Diagnostics (Government / GCC High)

Operating principles for diagnosing and remediating Intune compliance on a US sovereign-cloud tenant. They exist to prevent the two failure modes unique to GCC High: **silent wrong-cloud errors** and **chasing parity gaps as if they were bugs.**

Sovereign-cloud hygiene

- **Bookmark the `*.us` portals; delete the `.com` ones.** `intune.microsoft.us`, `portal.azure.us`,

`portal.office365.us`, `security.microsoft.us`, `compliance.microsoft.us`, `portal.manage.microsoft.us`. A `.com` bookmark is how a wrong-cloud diagnosis starts.

- **Always pass `-Environment USGov`** (DoD: `USGovDoD`) to `Connect-MgGraph`. Bake it into every

script and runbook; empty Graph output on GCC High is almost always a missing `-Environment`.

- **Never SSL-inspect `*.manage.microsoft.us`** or `*.dm.microsoft.us`. Add explicit proxy exclusions

and treat any new TLS-inspection rule as a change that can silently break compliance check-in.

Diagnose before you change

- **Start from the symptom, not the setting.** Use the Branch A–E tree; capture the exact state with

Discovery/Report before touching a policy.

- **Confirm the feature exists in GCC High first.** Before "fixing" a failing rule, check it against

`gcc-high-parity-notes.md`. A rule tied to an absent/planned Gov feature (MTD vendor not in Gov, classic Autopilot, an expanded DHA capability, Cloud PKI, Remote Help) can't be fixed — it must be removed or replaced.

- **One variable at a time.** Change a single setting, force sync, re-evaluate. Batching changes hides

which one fixed (or broke) compliance.

Safety and change control

- **Pilot every fix** on one device or a small group before expanding. A compliance change that flips a

fleet to **Not compliant** blocks every user simultaneously through Conditional Access.

- **Keep Remediation and Set-ScopeTargeting in `-WhatIf` until reviewed.** Read the planned actions;

execute only with an explicit `-Execute` (or `-WhatIf:$false`) after the pilot passes.

- **Expand in waves**, re-validating each wave. Confirm the escrow/attestation dependency (for DHA-backed rules) per wave.

Grace periods and Conditional Access

- **Set grace periods deliberately.** Too long hides real failures; too short blocks users mid-fix.
- **Fix CA problems in CA, compliance problems in compliance.** If a device is Compliant but blocked, the answer is in `portal.azure.us` Conditional Access + sign-in logs — not in the compliance policy.
- **Account for propagation lag.** A just-compliant device may need a sign-out/in before CA sees it.

Evidence and least privilege

- **Trust the portal as source of truth.** A script result isn't "done" until `intune.microsoft.us` agrees. Keep Discovery/Report output as before/after evidence for change records.
- **Use Intune Administrator, not Global Administrator**, for day-to-day diagnosis. Reserve GA for tenant-wide changes. The scripts run read-only (Discovery/Validation/Report) or preview-first (Remediation/Set-ScopeTargeting) by design — keep it that way.
- **Re-verify endpoints periodically.** The `*.us` wildcard hosts and IPs rotate as the service scales; re-check Microsoft's live US Government endpoints list rather than pinning stale IPs.

Documentation discipline

- **Record every parity substitution.** When you replace a Gov-unavailable rule, note what it replaced and why, so the next admin doesn't "restore" a rule that can't evaluate in GCC High.
- **Flag anything unverified.** If Microsoft doesn't explicitly confirm a behavior for GCC High, label it **"verify for GCC High"** in your runbook rather than assuming Commercial behavior.

8. Deliverables Summary — Compliance-Failure Diagnostics (Government / GCC High)

Everything included in the **Government (GCC High) edition** of the Intune Compliance-Failure Diagnostics & Remediation solution, and how the pieces fit together. All portals are `*.us`; all scripts connect to **US Gov Graph** (`Connect-MgGraph -Environment USGov`; DoD: `-Environment USGovDoD`).

Documents

Deliverable	What it gives you
01 — Solution Overview	What the solution does, the deployment model, and the **Commercial vs GCC High — what's different** table (portals, endpoints, feature parity).
Executive Summary	One-page business framing: the problem, the solution, the GCC High parity gaps that matter, and the value.
02 — Requirements Guide	Every prerequisite with **Verify / Implement**, confirmed in the `*.us` portals — including the US Gov service **endpoints** (and the no-SSL-inspection rule) and Graph-PowerShell `-Environment USGov` setup.
03 — Diagnostic Playbook	The core deliverable: a **symptom-driven decision tree** (Branches A–E) from reported symptom → exact GCC High blade → safe fix → validation, with GCC High parity notes inline at each step.
05 — Testing Guide	Nine pass/fail tests proving you're in the right cloud, endpoints are reachable/uninspected, scripts match the portal, fixes turn devices Compliant, and parity gaps are handled.
06 — Troubleshooting Guide	Failures of the **tooling/environment** (wrong cloud, TLS interception, stuck "Not evaluated," MTD/Autopilot parity gaps, propagation lag) with fixes.
07 — Best Practices	Operating principles: sovereign-cloud hygiene, diagnose-before-change, pilot/`-WhatIf` safety, CA-vs-compliance separation, evidence and least privilege.
GCC High Parity Notes	The **cited** reference for every Commercial-vs-GCC High difference, sourced to Microsoft US Government docs, with unconfirmed items flagged **"verify for GCC High."**
08 — Deliverables Summary	This document.

PowerShell scripts (US Gov Graph)

All scripts begin with a standard header (purpose, how-to-run, expected results) and use **US Gov Graph endpoints**. Read-only scripts make no changes; change scripts **default to** `-WhatIf` and require an explicit execute flag.

Script	Type	What it does
INTCOMPDXG-Discovery.ps1	Read-only	Snapshots a device/tenant: enrollment state, last check-in, applied compliance policies, and each failing setting — the starting point of the Playbook.

Script	Type	What it does
INTCOMPDXG-Validation.ps1	Read-only	Pass/fail check that a specific device (or group) now meets its assigned compliance policy after a fix.
INTCOMPDXG-Report.ps1	Read-only	Compliance rollup across a group/tenant (compliant / non-compliant / not-evaluated / grace) to confirm scope and match the portal.
INTCOMPDXG-Remediation.ps1	Change (`-WhatIf` **default**)	Previews then (with `-Execute`) applies safe remediation actions (e.g. trigger a compliance re-evaluation / sync, clear a stale grace state) for targeted devices.
INTCOMPDXG-Set-ScopeTargeting.ps1	Change (`-WhatIf` **default**)	Previews then (with `-Execute`) adds/removes a compliance policy's **group assignment** so you can pilot a fix on the affected population.

How they work together

1. **Assess** — `INTCOMPDXG-Discovery.ps1` + `INTCOMPDXG-Report.ps1` capture the exact symptom and its

spread.

1. **Diagnose** — the **Diagnostic Playbook** (03) routes the symptom to the correct `*.us` blade and

fix, confirming the rule's feature is **supported in GCC High** first.

1. **Pilot** — `INTCOMPDXG-Set-ScopeTargeting.ps1` (`-WhatIf` → `-Execute`) scopes the fix to a pilot

group.

1. **Validate** — `INTCOMPDXG-Validation.ps1` + the portal confirm the device turns **Compliant**.

2. **Remediate & Expand** — `INTCOMPDXG-Remediation.ps1` (`-WhatIf` → `-Execute`) applies at scale in

waves, re-validating each wave.

Success criteria

- You operate entirely in the `*.us` portals and **USGov** Graph environment.
- Affected devices move from **Not compliant / Not evaluated** to **Compliant**, and Conditional Access

unblocks the user.

• No device is left stuck on a **GCC High parity gap**; every change was previewed (`-WhatIf`) and piloted before expansion.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

INTCOMPDXG-Discovery.ps1

Purpose: Read-only starting point for the Diagnostic Playbook. For a device (by name) or a group, reports enrollment state, last check-in time, which compliance policies apply, the overall compliance state, and each failing setting — against Microsoft Graph US Gov endpoints. Pass your device name as -DeviceName 'your-device-name' or a group as -GroupName 'your-group'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Discovery.ps1 -DeviceName your-device-name (Graph scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS A per-device table: enrollment/last-check-in, overall compliance state, and PASS/FAIL per applied compliance policy/setting. Empty output almost always means you connected to the Commercial cloud — confirm (Get-MgContext).Environment is USGov. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$DeviceName, [string]$GroupName, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Discovery.ps1 -DeviceName your-device-name (Graph scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS A per-device table: enrollment/last-check-in, overall compliance state, and PASS/FAIL per applied compliance policy/setting. Empty output almost always means you connected to the Commercial cloud — confirm (Get-MgContext).Environment is USGov. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$DeviceName, [string]$GroupName, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

INTCOMPDXG-Remediation.ps1

Purpose: Applies low-risk remediation actions to targeted non-compliant devices: triggers a device sync (check-in) and a compliance re-evaluation so a device that was fixed on-box re-reports. Defaults to preview only (-WhatIf). Nothing changes until you re-run with -Execute. Target with -DeviceName 'your-device-name' or -GroupName 'your-group-name'. HOW TO RUN Preview (default, changes nothing): powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Remediation.ps1 -GroupName your-group-name Apply after a successful pilot: powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Remediation.ps1 -GroupName your-group-name -Execute (Graph scopes: DeviceManagementManagedDevices.ReadWrite.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS In preview: a list of the sync/re-evaluate actions that WOULD run per device, and no change. With -Execute: each action is sent and a PASS/FAIL per action is reported. Re-validate with INTCOMPDXG-Validation.ps1 after check-in. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints unless -Execute is supplied; change actions default to -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$DeviceName, [string]$GroupName, [switch]$Execute, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

How to run: Preview (default, changes nothing): powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Remediation.ps1 -GroupName your-group-name Apply after a successful pilot:

powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Remediation.ps1 -GroupName your-group-name -Execute (Graph scopes: DeviceManagementManagedDevices.ReadWrite.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS In preview: a list of the sync/re-evaluate actions that WOULD run per device, and no change. With -Execute: each action is sent and a PASS/FAIL per action is reported. Re-validate with INTCOMPDXG-Validation.ps1 after check-in. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints unless -Execute is supplied; change actions default to -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$DeviceName, [string]$GroupName, [switch]$Execute, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

INTCOMPDXG-Report.ps1

Purpose: Read-only. Produces a compliance rollup (compliant / noncompliant / inGracePeriod / notEvaluated / error / conflict) across the whole tenant or a named group, so you can see whether a failure is one device or a pattern and confirm the numbers match the portal. Optionally writes a CSV. Pass -GroupName 'your-group-name' to scope, or omit for tenant-wide. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Report.ps1 powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Report.ps1 -GroupName your-group-name -CsvPath .\compliance-report.csv (Graph scopes: DeviceManagementManagedDevices.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS A count by compliance state plus a per-device table. Counts should match Devices > Compliance in intune.microsoft.us. If they don't, confirm the environment (USGov) and the group scope. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$GroupName, [string]$CsvPath, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Report.ps1 powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Report.ps1 -GroupName your-group-name -CsvPath .\compliance-report.csv (Graph scopes: DeviceManagementManagedDevices.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS A count by compliance state plus a per-device table. Counts should match Devices > Compliance in intune.microsoft.us. If they don't, confirm the environment (USGov) and the group scope. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$GroupName, [string]$CsvPath, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

INTCOMPDXG-Set-ScopeTargeting.ps1

Purpose: Scopes a fix by adding (or removing) an Entra security group assignment on a device compliance policy, so you can pilot the corrected policy on the affected population before expanding. Defaults to preview only (-WhatIf); nothing changes until you re-run with -Execute. Provide the policy with -PolicyName 'your-policy-name' and the group with -GroupName 'your-group-name'; use -Action Add (default) or -Action Remove. HOW TO RUN Preview (default, changes nothing): powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Set-ScopeTargeting.ps1 -PolicyName your-policy-name -GroupName your-group-name Apply after review: powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Set-ScopeTargeting.ps1 -PolicyName your-policy-name -GroupName your-group-name -Execute (Graph scopes: DeviceManagementConfiguration.ReadWrite.All, Group.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS In preview: the current assignments and the single add/remove that WOULD be made, with no change. With

-Execute: the assignment is updated and the new assignment list is printed. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints unless -Execute is supplied; assignment change defaults to -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][string]$PolicyName, [Parameter(Mandatory=$true)][string]$GroupName, [ValidateSet('Add','Remove')][string]$Action = 'Add', [switch]$Execute, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

How to run: Preview (default, changes nothing): powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Set-ScopeTargeting.ps1 -PolicyName your-policy-name -GroupName your-group-name Apply after review: powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Set-ScopeTargeting.ps1 -PolicyName your-policy-name -GroupName your-group-name -Execute (Graph scopes: DeviceManagementConfiguration.ReadWrite.All, Group.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS In preview: the current assignments and the single add/remove that WOULD be made, with no change. With -Execute: the assignment is updated and the new assignment list is printed. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints unless -Execute is supplied; assignment change defaults to -WhatIf. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][string]$PolicyName, [Parameter(Mandatory=$true)][string]$GroupName, [ValidateSet('Add','Remove')][string]$Action = 'Add', [switch]$Execute, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

INTCOMPDXG-Validation.ps1

Purpose: Read-only. After you apply a fix, confirms a specific device (or the devices in a group) now meets each assigned compliance policy, and lists any setting still failing. Use this as the Validate step of the Playbook before expanding a fix. Pass -DeviceName 'your-device-name' (or -GroupName 'your-group-name'). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Validation.ps1 -DeviceName your-device-name (Graph scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS Per device: an overall PASS (Compliant) / FAIL verdict and, on FAIL, the specific setting states still failing. A device that just became compliant may need a reboot + sync first (especially DHA-backed BitLocker/Secure Boot/Code Integrity rules). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$DeviceName, [string]$GroupName, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')

How to run: powershell -ExecutionPolicy Bypass -File .\INTCOMPDXG-Validation.ps1 -DeviceName your-device-name (Graph scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementConfiguration.Read.All; connects with -Environment USGov. For DoD tenants pass -CloudEnvironment USGovDoD.) EXPECTED RESULTS Per device: an overall PASS (Compliant) / FAIL verdict and, on FAIL, the specific setting states still failing. A device that just became compliant may need a reboot + sync first (especially DHA-backed BitLocker/Secure Boot/Code Integrity rules). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #> [CmdletBinding()] param([string]$DeviceName, [string]$GroupName, [ValidateSet('USGov','USGovDoD')][string]$CloudEnvironment = 'USGov')